DALHOUSIE UNIVERSITY

Visualizing Data

Distributions, proportions, and associations

Prof. Gabriel Spadon, PhD

Faculty of Computer Science

spadon@dal.ca

1 Chart vocabulary

Every chart in this class rests on choices. Bins, bandwidths, units, denominators, groupings, and axis limits all change what a reader notices. A good caption states those choices and limits the claim to what the data support.

1.1 Start from the question

What it is for Table 1, the Class A map of chart families, gives candidates for each question, not a one-to-one rule.

Table 1. Chart families organized by their analytical question.

Question	Starting points
How much in each category?	Bars, dots, grouped or stacked bars, heatmaps.
How are values distributed?	Histograms, densities, box plots, violins, ECDFs, Q-Q plots.
How is a whole divided?	Bars, pies, stacked bars; stacked areas over time.
How do two variables relate?	Scatter plots; a line when order has meaning.
How do several variables relate?	Scatter-plot matrices, correlograms, parallel coordinates.

Two charts in one family can still answer different questions. A histogram and an ECDF both describe a distribution, but they make different comparisons easy. A bar chart and a pie both show proportions, but only the bar chart puts every part on a common baseline. The question narrows the choice; the audience and the sample size decide the final design.

2 Example data and a reproducible plotting session

The data Iris contains 150 flowers, 50 from each of setosa, versicolor, and virginica. Its four numeric columns are sepal length, sepal width, petal length, and petal width, all in centimetres [4, 11]. The copy that ships with R and the copy in scikit-learn agree on all 600 numeric entries, so no download is needed. These data illustrate plotting choices. They do not show how every iris population is distributed.

The setup Install any missing package once, with `install.packages(c("ggplot2", "GGally"))` in R or `pip install numpy pandas matplotlib seaborn scipy scikit-learn` in Python. Then run the setup once and the examples below in order. In an interactive session, print an R plot object, such as `print(p_histogram)`, or call `plt.show()` in Python to see it. Real course data still need checks for types, units, missing values, and reporting limits before plotting.

R

```r
library(ggplot2)
library(GGally)
d <- iris
species <- levels(d$Species)
colours <- c("#1F5C8B", "#A85400", "#2E7D5B")
styles <- c("solid", "dashed", "dotted")
h_group <- 0.20  # common bandwidth, cm
numeric_names <- names(d)[1:4]
```

Python

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import norm
from sklearn.datasets import load_iris
raw = load_iris(as_frame=True)
d = raw.data.copy()
d.columns = ["Sepal.Length", "Sepal.Width",
             "Petal.Length", "Petal.Width"]
d["Species"] = pd.Categorical.from_codes(
    raw.target, raw.target_names)
species = list(raw.target_names)
colours = ["#1F5C8B", "#A85400", "#2E7D5B"]
styles = ["-", "--", ":"]
markers = ["o", "^", "s"]
numeric_names = list(d.columns[:4])
h_group = 0.20  # common bandwidth, cm
def h_factor(kde, h=h_group):
    # SciPy scales the factor by the SD.
    return h / kde.dataset.std(ddof=1)
```

3 Visualizing one-variable distributions

3.1 Box plots show quartiles, fences, and whiskers

The box, the $1.5\,\mathrm{IQR}$ fences and the whiskers follow the Class 02 definitions [17].

Worked example For Iris petal length the summary is $(1.0, 1.6, 4.35, 5.1, 6.9)$ cm, so $\mathrm{IQR} = 5.1 - 1.6 = 3.5$ cm and the fences are $1.6 - 5.25 = -3.65$ cm and $5.1 + 5.25 = 10.35$ cm. No value lies outside them, so here the whiskers happen to reach the minimum and maximum. The gap between Q_1 and the median is much wider than the gap below Q_1. That describes spacing only. It does not show how many groups produced the data; a histogram or density adds shape.

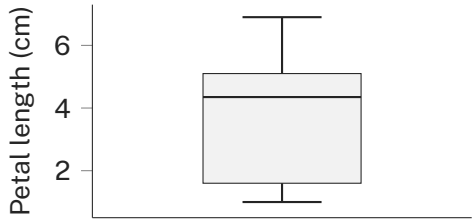


Figure 1. Petal-length box plot using linearly interpolated quartiles and the 1.5-IQR whisker rule. All plotted values are in centimetres.

R

```r
x <- d$Petal.Length
print(quantile(x, c(0, .25, .5, .75, 1)))
p_box <- ggplot(d, aes(y = Petal.Length)) +
  geom_boxplot(coef = 1.5) +
  labs(y = "Petal length (cm)")
```

Python

```python
x = d["Petal.Length"]
print(x.quantile([0, .25, .5, .75, 1]))
fig, ax = plt.subplots()
sns.boxplot(y=x, whis=1.5, ax=ax)
ax.set_ylabel("Petal length (cm)")
```

Differences to know

- The examples use ggplot2's type-7 quantiles and pandas' linear interpolation, which agree; the type numbers follow Hyndman and Fan [8]. Base R's `boxplot` uses Tukey hinges, which can differ from these quartiles on other samples.
- Record the convention when a small difference in a box edge or a flagged point matters.

3.2 Histograms make binning explicit

What it is for A histogram shows the shape of a numeric variable by counting values in intervals.

The rule A histogram divides the numeric axis into bins and counts the observations in each (Figure 2). A bar chart compares named categories; a histogram's bins are chosen by the analyst.

- With equal-width bins, bar height can show the count.
- With unequal widths, height must be count divided by width, so that bar *area* represents count.
- A probability-density histogram also divides by the total count, so the total area is one.

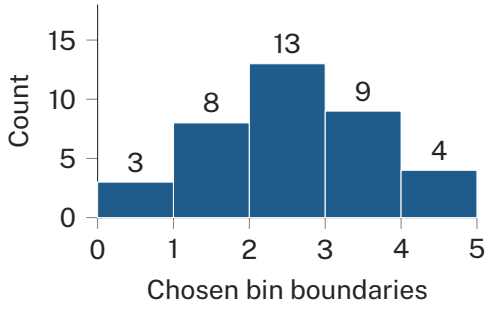


Figure 2. Five equal-width bins with counts 3, 8, 13, 9, and 4. Bar heights are calculated directly from their labelled counts.

Worked example Figure 3 draws the same 150 sepal lengths over the interval 4.25–8.25 cm, with either 30 or 10 equal-width bins. The widths are $4/30 \approx 0.133$ cm and $4/10 = 0.4$ cm. The ten counts are 9, 23, 20, 28, 19, 23, 16, 6, 5, and 1, which sum to 150. Measurements are recorded to 0.1 cm, so a grid near that resolution turns the recording pattern into spikes. Moving an edge or changing the width can change the apparent shape without changing any observation. Neither view is the one true distribution of the flowers.

R

```r
edges <- seq(4.25, 8.25, by = 0.4)
bins <- cut(d$Sepal.Length, edges,
  right = FALSE, include.lowest = TRUE)
print(table(bins))
p_histogram <- ggplot(d, aes(Sepal.Length)) +
  geom_histogram(
    breaks = edges, closed = "left") +
  labs(x = "Sepal length (cm)", y = "Count")
```

Python

```python
edges = np.linspace(4.25, 8.25, 11)
counts, _ = np.histogram(
    d["Sepal.Length"], bins=edges)
print(counts)
fig, ax = plt.subplots()
ax.hist(d["Sepal.Length"], bins=edges)
ax.set(xlabel="Sepal length (cm)",
       ylabel="Count")
```

Edges and endpoints The edges lie between the observed tenth-centimetre values, so no observation sits on a boundary. Both examples use left-closed intervals, and NumPy also includes the final

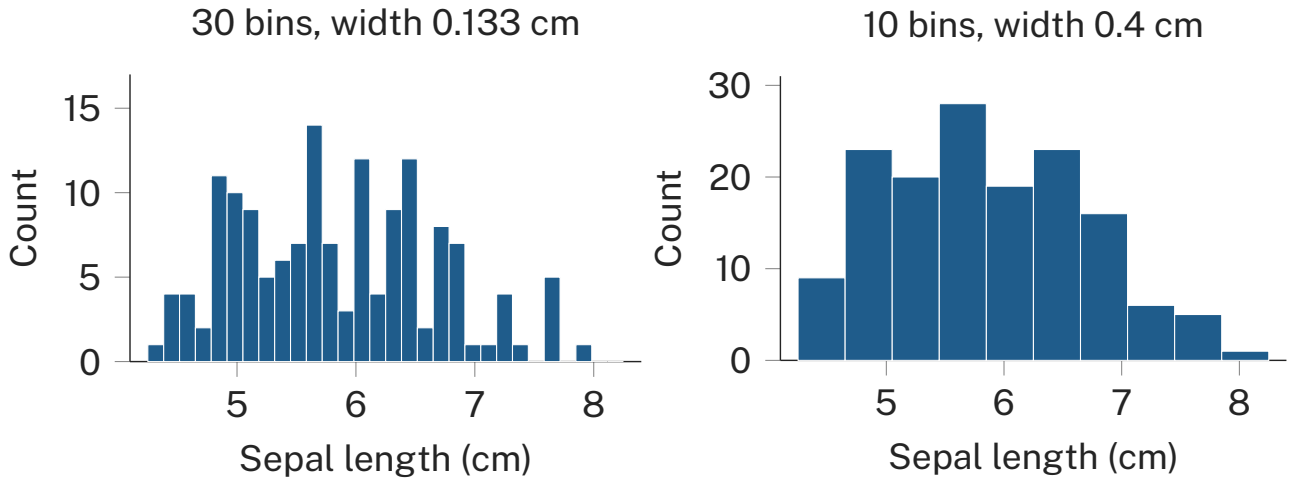


Figure 3. The same 150 sepal lengths under two explicit grids. The 10-bin view has counts 9, 23, 20, 28, 19, 23, 16, 6, 5, and 1, which sum to 150.

right edge in the last bin. No observation lies on that edge here. For another dataset, state and test endpoint membership directly. A bin count or a width alone does not fix a histogram; the origin, the full edge vector, and the closure rule matter too.

Differences to know

- Base R's `hist` picks convenient boundaries after Sturges' rule [16], so it can return a different number of bins even when an integer is requested.
- NumPy's `auto`, also used by seaborn, takes the narrower of the Freedman–Diaconis [5] and Sturges widths. For these sepal lengths NumPy uses 9 bins under `auto` and 8 under `fd`.
- Explicit edges make a comparison reviewable.

3.3 Nearly equal summaries need not mean equal shape

What it is for This example shows what a five-number summary leaves out, and why a view of the observations belongs next to it.

Worked example Figure 4 uses three datasets from the box-plot examples of Matejka and Fitzmaurice [10]. Each has 2,484 observations and a five-number summary close to $(-9.77, -2.69, 0, 2.68, 9.76)$. The summaries are nearly, not exactly, equal under the linear-interpolation convention. Histograms on one shared grid still show very different concentrations of values.

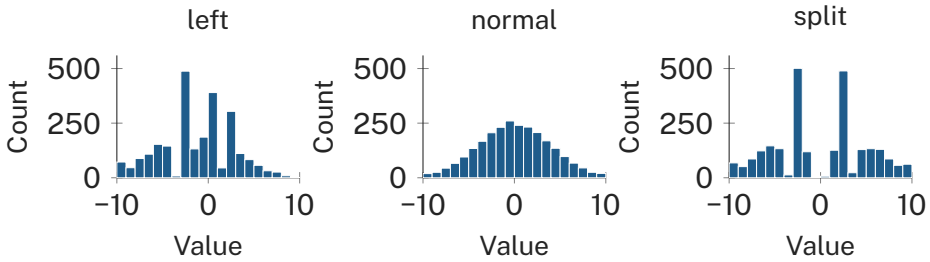


Figure 4. Source datasets named left, normal, and split, drawn from the actual observations with one shared histogram grid. The names identify the source sets; they are not diagnoses inferred from a box plot.

The observations are the datasauRus box-plot data, distributed in the package [6] under its MIT licence, and the figures are computed from them. Check a view of the observations alongside any compact summary.

3.4 Kernel density estimates average smooth contributions

What it is for A kernel density estimate draws the shape of a distribution as a smooth curve, without bin edges.

The formula Place a smooth kernel (here a bell-shaped Gaussian curve) at each observation and average the kernels. With a kernel K whose integral is one,

$$\widehat{f}_h(t) = \frac{1}{nh} \sum_{i=1}^{n} K\Big(\frac{t - x_i}{h}\Big).$$

- n is the number of observations and x_i is observation i.
- t is the location where the curve is evaluated.
- K is the standard-normal density shape.
- The bandwidth $h > 0$ sets each kernel's width and has the unit of the measurement.

Each contribution has area $1/n$, so the whole curve has area one. For widths in centimetres, density has unit cm^{-1}. A density value is not the probability of one exact width; probabilities are areas over intervals.

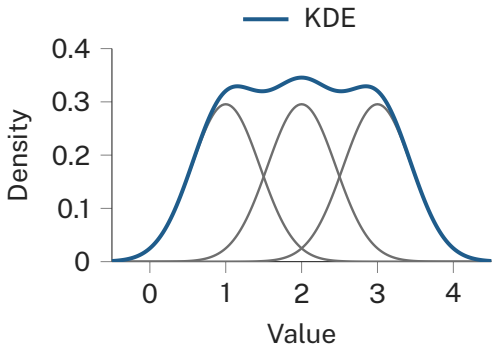


Figure 5. Three Gaussian contributions with observations 1, 2, and 3 and bandwidth 0.45. Each thinner curve has area one third; their sum is the thicker density curve with total area one.

Bandwidth A small bandwidth keeps more local variation, including sampling noise. A large bandwidth smooths more and can merge distinct features.

Worked example Figure 6 uses the fixed bandwidth $h = 0.25$ cm for petal width. It shows a concentration at low widths and a broader region at higher widths. The Gaussian kernels also put some density below zero, although a width cannot be negative. This boundary artefact belongs to the unconstrained estimator.

How the code matches the formula In R, `sapply` evaluates the sum at each grid point and `dnorm` returns one Gaussian contribution per observation. In Python, `grid[:, None]` makes a column of

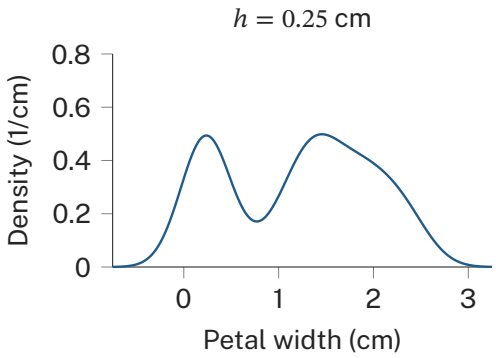


Figure 6. Petal-width Gaussian KDE with bandwidth 0.25 cm and a common evaluation grid. The curve is normalized over the full real line; the figure displays the stated finite range.

grid points. Combined with the observation vector, it gives a table with one row per grid point and one column per observation, and `mean(axis=1)` averages each row. With the same grid, formula, and bandwidth, the two curves match.

R

```r
h <- 0.25  # bandwidth in cm
x <- d$Petal.Width
grid <- seq(-0.75, 3.25, length.out = 201)
f <- sapply(grid, function(t) {
  mean(dnorm(t, x, h))
})
curve <- data.frame(width = grid, density = f)
p_density <- ggplot(curve,
  aes(width, density)) + geom_line() +
  labs(x = "Petal width (cm)",
       y = "Density (1/cm)")
```

Python

```python
h = 0.25  # bandwidth in cm
x = d["Petal.Width"].to_numpy()
grid = np.linspace(-0.75, 3.25, 201)
kernels = norm.pdf(grid[:, None],
                   loc=x, scale=h)
f = kernels.mean(axis=1)
fig, ax = plt.subplots()
ax.plot(grid, f)
ax.set(xlabel="Petal width (cm)",
       ylabel="Density (1/cm)")
```

Differences to know

- R's default `nrd0` bandwidth is $0.9\,\min(s, \mathrm{IQR}/1.34)\,n^{-1/5}$, Silverman's rule of thumb [14]. SciPy's Scott rule [13] is $s\,n^{-1/5}$ for one-dimensional data.
- For these petal widths the two defaults give about 0.251834 cm and 0.279816 cm. Both curves remain bimodal. The formulas, not the languages, explain the difference.
- Some APIs take a bandwidth in measurement units. SciPy's `bw_method` takes a factor that multiplies the sample SD, so a fixed physical bandwidth h is passed as h/s.

4 Comparing distributions between groups

This section applies the charts of Section 3 to three species at once. The same methods work when group sizes differ, although all three Iris species have 50 records.

4.1 Stacked histograms show totals

What it is for A stacked histogram answers how many observations, from all groups together, fall in each interval.

The rule Each bin stacks the groups' counts. Only the bottom group starts at the common zero baseline, so comparing an interior group's shape means following segment boundaries that move

from bin to bin. A stack suits contributions to a total. To compare within-group shapes, use facets, normalized densities, boxes, or violins.

Worked example Figure 7 stacks sepal length by species with the same 0.4-cm bins for every species.

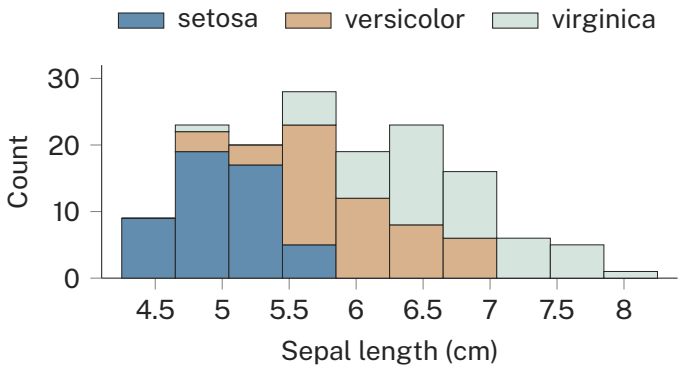


Figure 7. Species counts stacked using the same 0.4-cm bins. Stack order follows the explicit species order in both examples.

R

```r
# Fills differ in lightness as well as hue.
fills <- c("#628DAE", "#D8B28C", "#D5E5DE")
p_group_hist <- ggplot(d,
  aes(Sepal.Length, fill = Species)) +
  geom_histogram(
    breaks = edges, closed = "left",
    position = position_stack(reverse = TRUE)) +
  scale_fill_manual(values = fills) +
  labs(x = "Sepal length (cm)", y = "Count")
```

Python

```python
# Fills differ in lightness as well as hue.
fills = ["#628DAE", "#D8B28C", "#D5E5DE"]
fig, ax = plt.subplots()
groups = [
    d.loc[d["Species"] == s, "Sepal.Length"]
    for s in species]
ax.hist(groups, bins=edges, stacked=True,
        color=fills, label=species)
ax.set(xlabel="Sepal length (cm)",
       ylabel="Count")
ax.legend()
```

4.2 Normalized group densities compare shape

What it is for One density per group compares the shapes of the groups regardless of their sizes.

The rule Normalize each curve to area one. The curves then compare shape, but no longer show which group has more records, so report the group counts as well. Use one shared bandwidth so that the amount of smoothing is the same for every group.

Worked example Figure 8 uses a shared bandwidth of 0.20 cm. Setosa has shorter sepal lengths overall, but the ranges and densities of the three species overlap.

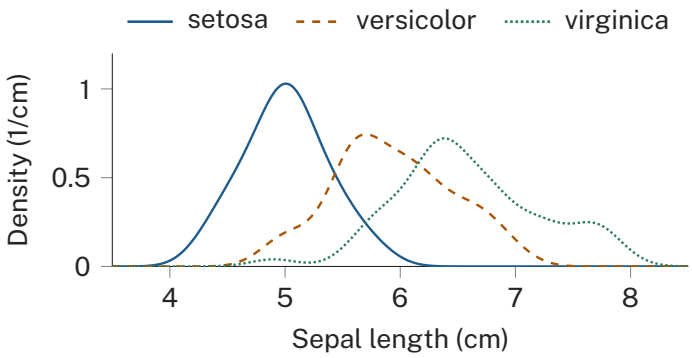


Figure 8. Gaussian group densities, each normalized to area one before display, with a shared bandwidth of 0.20 cm. Colour and line pattern identify species.

R

```r
p_group_density <- ggplot(d,
  aes(Sepal.Length, colour = Species,
      linetype = Species)) +
  geom_density(bw = h_group) +
  scale_colour_manual(values = colours) +
  scale_linetype_manual(values = styles) +
  labs(x = "Sepal length (cm)",
       y = "Density (1/cm)")
```

Python

```python
fig, ax = plt.subplots()
for values, s, c, line in zip(
        groups, species, colours, styles):
    factor = h_group / values.std(ddof=1)
    sns.kdeplot(
        values, bw_method=factor,
        common_norm=False, color=c,
        linestyle=line, label=s, ax=ax)
ax.set(xlabel="Sepal length (cm)",
       ylabel="Density (1/cm)")
ax.legend()
```

Differences to know

- Python passes h/s for each group because SciPy multiplies the factor by that group's SD. The bandwidth stays 0.20 cm in both languages.
- Seaborn's grouped `kdeplot` defaults to `common_norm=True`, which weights each curve by its group's share of the observations. With three equal groups, each curve then has area one third. The examples normalize each group separately instead.
- R and Python use slightly different evaluation grids and numerical KDE algorithms, so coordinates can differ slightly with the same estimator and bandwidth. The printed figures evaluate the Gaussian formula directly on a common grid.

4.3 Grouped box plots compare location and spread

What it is for Side-by-side boxes compare location, spread, and flagged points across groups in little space.

Worked example Figure 9 and Table 2 use linear-interpolation quartiles for all species. Virginica has $Q_1 = 6.225$ cm. Its 4.9-cm observation lies below the lower fence, and its lower whisker ends at 5.6 cm. The specimen may be unusual or misrecorded, and the box plot cannot say which. Keep the distinction between flagging an observation and changing it.

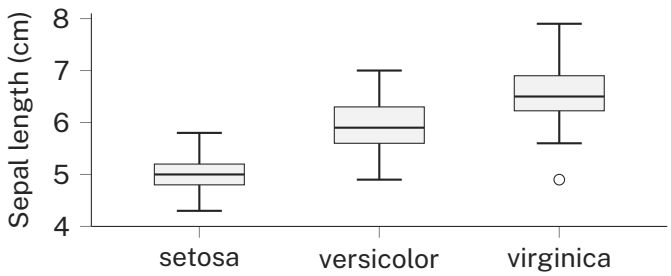


Figure 9. Sepal-length box plots using one quartile and whisker convention for all species. The low virginica point remains visible.

Table 2. Sepal-length box-plot summaries in centimetres. Whisker endpoints can differ from sample extrema.

Species	Lower whisker	Q_1	Median	Q_3	Upper whisker
setosa	4.3	4.8	5.0	5.2	5.8
versicolor	4.9	5.6	5.9	6.3	7.0
virginica	5.6	6.225	6.5	6.9	7.9

R

```r
p_group_box <- ggplot(d,
  aes(Species, Sepal.Length)) +
  geom_boxplot(coef = 1.5, fill = "#F2F2F2") +
  labs(x = "Species", y = "Sepal length (cm)")
```

Python

```python
fig, ax = plt.subplots()
sns.boxplot(
    data=d, x="Species", y="Sepal.Length",
    order=species, whis=1.5,
    color="#F2F2F2", saturation=1, ax=ax)
ax.set_ylabel("Sepal length (cm)")
```

Report the convention when exact box edges matter. Sample quartiles, Tukey hinges, whiskers, and fences are related but distinct quantities.

4.4 Violins add estimated shape

What it is for A violin, introduced by Hintze and Nelson [7], shows each group's estimated density, not only its quartiles.

The rule A violin mirrors a density curve around its category position, so the two sides are identical by construction. The measurement axis runs vertically here, and skewness is read along it, not as a left-right imbalance. Width at a given value shows relative estimated density under the chosen scaling.

Worked example Figure 10 uses a Gaussian bandwidth of 0.20 cm and equal-area scaling applied before trimming. Tails are cut at each species' observed range, so the visible areas can differ after trimming. The examples omit an internal box so the density boundary is easier to see. For a very small group, show the observations directly and treat a violin's shape with caution.

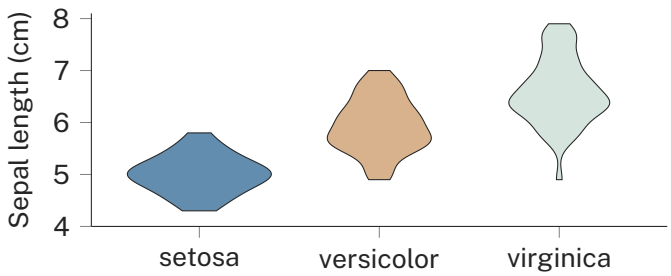


Figure 10. Sepal-length violins with Gaussian bandwidth 0.20 cm, equal-area scaling before trimming, and tails restricted to each species' observed range. Visible areas can differ after trimming.

R

```r
p_violin <- ggplot(d,
  aes(Species, Sepal.Length, fill = Species)) +
  geom_violin(bw = h_group, trim = TRUE,
              scale = "area") +
  scale_fill_manual(values = fills) +
  labs(x = "Species", y = "Sepal length (cm)")
```

Python

```python
fig, ax = plt.subplots()
sns.violinplot(
    data=d, x="Species", y="Sepal.Length",
    hue="Species", order=species,
    palette=fills, saturation=1,
    legend=False, dodge=False,
    bw_method=h_factor, cut=0,
    density_norm="area", common_norm=True,
    inner=None, ax=ax)
ax.set_ylabel("Sepal length (cm)")
```

Differences to know

- The Python callback receives each group's density estimator and returns h/s from that group's measurements. This fixes the physical bandwidth while letting the groups have different SDs.
- Seaborn defaults to `density_norm="area"`, not equal maximum width, and to `cut=2`, which extends tails beyond the observed range. ggplot2 trims its violins by default.
- Matching the trimming rule alone does not make the two plots match if their bandwidths or group scaling differ.

Boxes and violins together Figure 11 returns to all five source box-plot datasets. The narrow internal boxes are almost identical, while the density shapes differ. The bandwidth is $h = 0.6$ in the dataset's value units. That choice is part of this illustration, not a general rule.

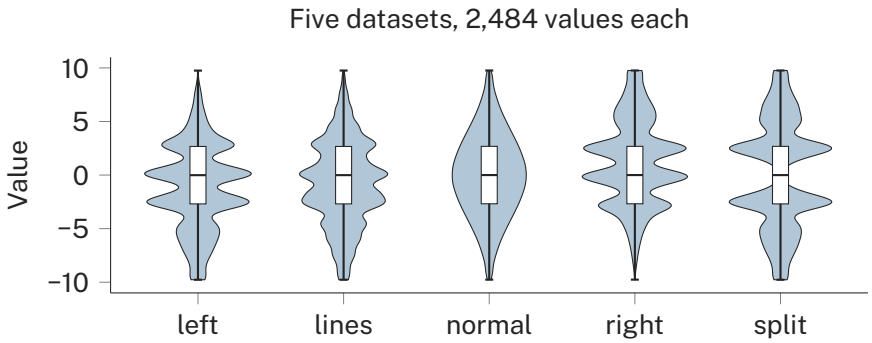


Figure 11. Five datasets from Matejka and Fitzmaurice, distributed through datasauRus. Each has 2,484 observations and nearly equal five-number summaries; the density shapes show what those summaries omit.

Both languages read the pinned source file, check that each set has 2,484 values, and draw the violins with $h = 0.6$ and a box inside each.

R

```r
url <- paste0(
  "https://raw.githubusercontent.com/",
  "jumpingrivers/datasauRus/",
  "0496ac15208e9ee4a58ea81a5de46912f095aa15/",
  "inst/extdata/Boxplots-Long.tsv")
same <- read.delim(url)
print(table(same$Plot))  # 2484 in each set
p_same <- ggplot(same, aes(Plot, Values)) +
  geom_violin(bw = 0.6, trim = TRUE,
              scale = "area") +
  geom_boxplot(width = 0.1,
               outliers = FALSE) +
  labs(x = NULL, y = "Value")
```

Python

```python
url = ("https://raw.githubusercontent.com/"
       "jumpingrivers/datasauRus/"
       "0496ac15208e9ee4a58ea81a5de46912f095aa15/"
       "inst/extdata/Boxplots-Long.tsv")
same = pd.read_csv(url, sep="\t")
print(same["Plot"].value_counts())  # 2484 each
fig, ax = plt.subplots()
sns.violinplot(
    data=same, x="Plot", y="Values",
    bw_method=lambda kde: h_factor(kde, 0.6),
    cut=0, density_norm="area", inner="box",
    ax=ax)
ax.set(xlabel=None, ylabel="Value")
```

5 Cumulative distributions and quantile comparisons

5.1 The ECDF counts values at or below a threshold

What it is for The empirical cumulative distribution function gives, for any threshold, the fraction of observations at or below it. It needs no bins and no bandwidth.

The definition For observations $x_1, \ldots, x_n$,

$$\widehat{F}(t) = \frac{\#\{x_i \le t\}}{n}.$$

- It is right-continuous, because an observation equal to t is counted.
- Each distinct value produces a jump. If m records are tied at a value, the jump there is m/n.
- It is zero below the smallest value and one at and above the largest.
- A flat section has no observations inside it.

Small example For the observations $1, 2, 2, 4$, the jumps are $1/4$ at 1, $2/4$ at 2 (two tied values), and $1/4$ at 4 (Figure 12).

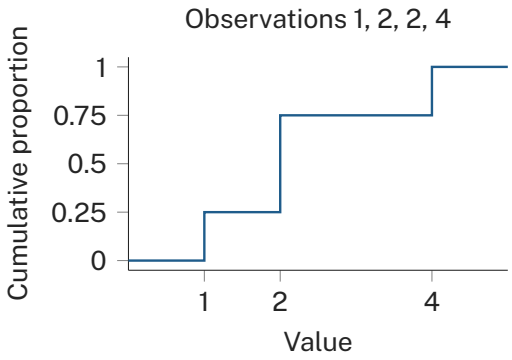


Figure 12. The observations 1, 2, 2, and 4 produce jumps of 0.25, 0.50, and 0.25. The combined jump at 2 accounts for the tie.

An ECDF works for discrete and continuous measurements. It still depends on which records were selected, any weights, and whether the measurements are valid. A missing value is not a numeric threshold, and a censored reporting limit is not an exact value. Report how many records were eligible and how they were treated. An ECDF describes the observed sample; a claim about a population needs a sampling argument.

Worked example For the 150 Iris petal widths, $\widehat{F}(1.3) = 78/150 = 0.52$, so 78 recorded widths are at or below 1.3 cm. Ties make jumps larger than $1/150$. Reading an ECDF works in two directions.

- Vertically, at one measurement, it gives a cumulative proportion.
- Horizontally, at one proportion, it gives a sample quantile.

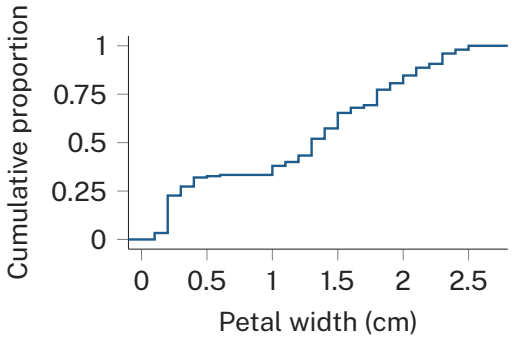


Figure 13. Petal-width ECDF with both its lower zero tail and upper one tail visible. Ties create jumps larger than 1/150.

R

```r
x <- d$Petal.Width
F_hat <- ecdf(x)
print(F_hat(1.3))
p_ecdf <- ggplot(d, aes(Petal.Width)) +
  stat_ecdf() +
  labs(x = "Petal width (cm)",
       y = "Cumulative proportion")
```

Python

```python
x = d["Petal.Width"].to_numpy()
print(np.mean(x <= 1.3))
fig, ax = plt.subplots()
sns.ecdfplot(x=x, ax=ax)
ax.set(xlabel="Petal width (cm)",
       ylabel="Cumulative proportion")
```

Building it from counts Count each distinct value, accumulate the counts with `cumsum`, and divide by n. The assertions check this against the threshold calculation. To draw the result by hand, use a right-continuous step and extend flat tails at zero below the minimum and at one above the maximum. Joining the distinct values with sloped lines would imply changes between observations that never happened.

R

```r
values <- sort(unique(x))
counts <- tabulate(match(x, values))
proportion <- cumsum(counts) / length(x)
print(data.frame(values, counts, proportion))
stopifnot(all.equal(proportion, F_hat(values)))
```

Python

```python
values, counts = np.unique(
    x, return_counts=True)
proportion = np.cumsum(counts) / len(x)
print(pd.DataFrame({
    "values": values, "counts": counts,
    "proportion": proportion}))
direct = [(x <= v).mean() for v in values]
assert np.allclose(proportion, direct)
```

5.2 Comparing groups on one cumulative scale

What it is for Grouped ECDFs compare entire distributions at every threshold at once.

Worked example Every ECDF ends at one, whatever the group size. In Figure 14, setosa reaches one before versicolor begins. Versicolor's ECDF is at or above virginica's throughout, and the two do not cross, so in this sample versicolor petal widths sit toward smaller values. The sample does not prove the same ordering for the whole population.

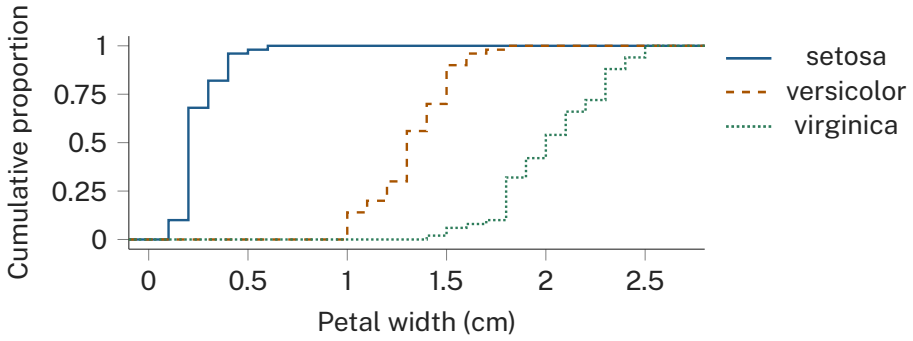


Figure 14. Species-specific petal-width ECDFs with a common scale. Distinct line patterns and an external legend preserve group identification where steps overlap.

R

```r
p_group_ecdf <- ggplot(d,
  aes(Petal.Width, colour = Species,
      linetype = Species)) +
  stat_ecdf() +
  scale_colour_manual(values = colours) +
  scale_linetype_manual(values = styles) +
  labs(x = "Petal width (cm)",
       y = "Cumulative proportion")
```

Python

```python
fig, ax = plt.subplots()
for s, c, line in zip(
        species, colours, styles):
    values = d.loc[d["Species"] == s,
                   "Petal.Width"]
    sns.ecdfplot(values, color=c,
                 linestyle=line, label=s,
                 ax=ax)
ax.set(xlabel="Petal width (cm)",
       ylabel="Cumulative proportion")
ax.legend()
```

Keep the same axes for every group. When grouped ECDFs cross, no single ordering holds at all thresholds. Step curves can also coincide and hide each other, so use distinct line patterns, direct labels, or separate aligned panels.

5.3 Q-Q plots compare a sample with reference quantiles

What it is for A normal Q-Q plot checks whether a sample has a normal shape.

The definition A quantile is the threshold for a chosen cumulative proportion. The plot puts ordered sample values on one axis and normal quantiles (mean zero, SD one) on the other [19].

- The plotting positions are $p_i = (i - 0.5)/n$ for $i = 1, \ldots, n$.
- The horizontal values are the normal quantiles at those positions, from `qnorm` in R or `norm.ppf` in Python.
- The reference line passes through the sample's first and third quartiles.

A sample with a normal shape after shifting and rescaling gives an approximately straight pattern, with some sampling scatter. Systematic bends or tail departures suggest a normal approximation would miss features of the sample.

Worked example Figure 15 shows petal width against the normal reference, with a quartile line of slope 1.11195 cm and intercept 1.05 cm. The points depart from the line. The bounded, grouped measurements explain why this is plausible, but the Q-Q shape alone cannot identify a number of mixture components or a cause.

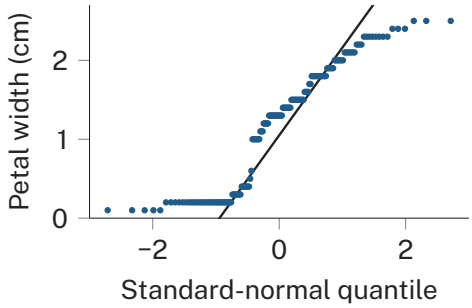


Figure 15. Iris petal-width normal Q-Q plot using the stated plotting positions and a quartile reference line with slope 1.11195 cm and intercept 1.05 cm.

R

```r
n <- length(x)
q <- qnorm((seq_len(n) - 0.5) / n)
quartiles <- quantile(x, c(.25, .75))
slope <- diff(quartiles) /
  diff(qnorm(c(.25, .75)))
intercept <- quartiles[1] - slope * qnorm(.25)
qq_data <- data.frame(q = q, value = sort(x))
p_qq <- ggplot(qq_data, aes(q, value)) +
  geom_point() +
  geom_abline(intercept = intercept,
              slope = slope) +
  labs(x = "Standard-normal quantile",
       y = "Petal width (cm)")
```

Python

```python
n = len(x)
prob = (np.arange(1, n + 1) - 0.5) / n
q = norm.ppf(prob)
quartiles = np.quantile(x, [.25, .75])
normal_iqr = np.diff(norm.ppf([.25, .75]))[0]
slope = np.diff(quartiles)[0] / normal_iqr
intercept = quartiles[0] - slope * norm.ppf(.25)
fig, ax = plt.subplots()
ax.scatter(q, np.sort(x))
ax.plot(q, intercept + slope * q)
ax.set(xlabel="Standard-normal quantile",
       ylabel="Petal width (cm)")
```

R's `qqnorm`, SciPy's `probplot`, and statsmodels' `qqplot` are shortcuts, but they can use other plotting positions and reference lines. The explicit examples remove those differences.

6 Visualizing proportions

6.1 A proportion needs a denominator

What it is for A proportion shows one part's share of a defined whole. Define the whole before drawing the parts.

The rule A proportion is a count divided by the total. Changing the denominator changes the question.

Worked example An assembly has 157, 64, 73, and 26 seats for parties A, B, C, and D. The total is 320. The proportions are 0.490625, 0.2, 0.228125, and 0.08125, or 49.0625%, 20%, 22.8125%, and

8.125%, and they sum to one. Every graphic in this section uses this vector; labels are rounded only for display.

R

```r
parts <- data.frame(
  Party = factor(LETTERS[1:4],
                 levels = LETTERS[1:4]),
  Count = c(157, 64, 73, 26))
parts$Prop <- parts$Count / sum(parts$Count)
print(parts)
p_proportions <- ggplot(parts,
  aes(Party, Prop)) +
  geom_col() + labs(y = "Proportion")
```

Python

```python
parts = pd.DataFrame({
    "Party": list("ABCD"),
    "Count": [157, 64, 73, 26]})
total = parts["Count"].sum()
parts["Prop"] = parts["Count"] / total
print(parts)
ax = parts.plot.bar(
    x="Party", y="Prop", legend=False)
ax.set_ylabel("Proportion")
```

6.2 Pies and bars

The rule A pie encodes shares as angles or areas. A bar chart encodes them as lengths from a shared zero baseline. Lengths on a common baseline are easier to compare precisely [2].

Worked example In Figure 16, A and D are clearly different in either view. The small difference between B (20%) and C (22.8%) is easier to see on the aligned bars. This is a choice made for a task, not a ban on pies.

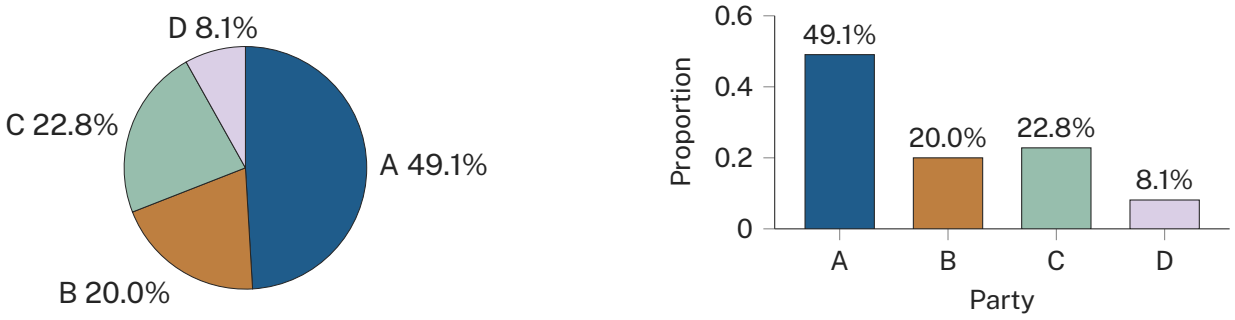


Figure 16. The same seat shares as a pie and separate bars. Labels are rounded to one decimal percentage point; exact proportions determine the geometry.

6.3 Stacked bars emphasize composition

What it is for A stack shows how one whole divides into parts.

The rule A single stack splits the whole into adjacent segments whose heights sum to one. The interior segments do not share a baseline. Several normalized stacks can compare compositions between groups or dates, but not their absolute sizes. Two groups can have identical shares and very different totals, so give the totals separately when readers need both. A stacked area chart extends the stack over an ordered time axis; it needs the same care with denominators and interior baselines.

R

```r
p_stacked <- ggplot(parts,
  aes(x = "All", y = Prop, fill = Party)) +
  geom_col(
    position = position_stack(reverse = TRUE)) +
  labs(x = NULL, y = "Proportion")
p_pie <- p_stacked + coord_polar(theta = "y") +
  theme_void()
```

Python

```python
one_row = parts.set_index("Party")[["Prop"]].T
stacked_ax = one_row.plot.bar(stacked=True)
stacked_ax.set(xlabel="", ylabel="Proportion")
fig, ax = plt.subplots()
ax.pie(parts["Count"], labels=parts["Party"],
       startangle=90, counterclock=False)
```

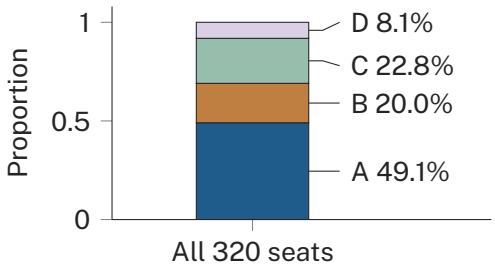


Figure 17. One stack of all four party shares. A is at the bottom, followed by B, C, and D, matching the explicit order in the examples.

Differences to know

- A pandas Series is one-dimensional, so transposing it does not make one stacked row. The Python example selects a one-column DataFrame and transposes that, giving one row with four party columns.
- The R stack sets its order with an explicit option.
- Pie orientation differs between libraries without changing any share. Identify categories in a legend or with direct labels.

Choosing the encoding Figure 18 compares the three encodings. No one chart is best for every task. Add labels or exact values when readers need precise comparisons.

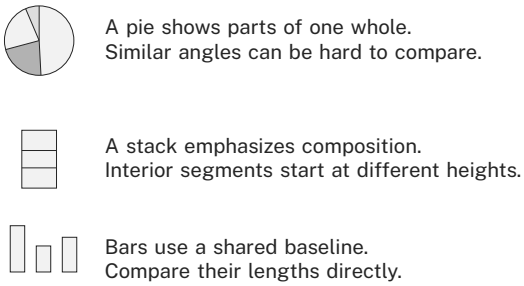


Figure 18. Pie angles, stacked segment lengths, and lengths on a common baseline support different comparison tasks. Direct labels can make exact shares easier to recover.

7 Visualizing associations

7.1 A labelled scatter plot

What it is for A scatter plot shows how two numeric variables vary together, one point per observation.

Worked example Figure 19 puts petal length on the horizontal axis and sepal width on the vertical axis. Each point is one flower. Species is shown by both colour and marker shape. This redundant encoding helps readers who cannot tell the colours apart and keeps a greyscale copy readable. In ggplot2, a colour mapped inside aes encodes a variable and creates a legend; a colour set outside aes is a constant style [18].

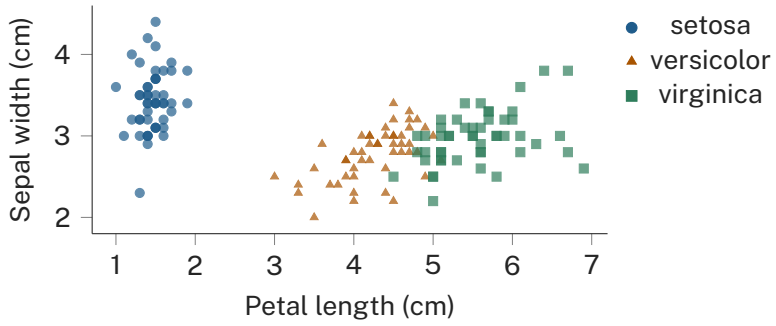


Figure 19. All 150 flowers with species shown by colour and shape. Both axes are measurements in centimetres, and the legend sits outside the data area.

R

```r
p_scatter <- ggplot(d,
  aes(Petal.Length, Sepal.Width,
      colour = Species, shape = Species)) +
  geom_point(alpha = .7) +
  scale_colour_manual(values = colours) +
  scale_shape_manual(values = c(16, 17, 15)) +
  labs(x = "Petal length (cm)",
       y = "Sepal width (cm)")
```

Python

```python
fig, ax = plt.subplots()
sns.scatterplot(
    data=d, x="Petal.Length", y="Sepal.Width",
    hue="Species", style="Species",
    hue_order=species, style_order=species,
    markers=markers, palette=colours,
    alpha=.7, ax=ax)
ax.set(xlabel="Petal length (cm)",
       ylabel="Sepal width (cm)")
```

Transparency reveals points that coincide, but the darkness it produces is not an exact density. Large samples may need smaller markers, facets, or a count-based display. Do not narrow an axis range to hide part of a relationship without saying so.

7.2 Marker area for a fourth variable

What it is for Marker area can carry one more numeric variable while position still shows the original pair.

The rule Map area, not radius, in proportion to the value, on a scale starting at zero. Within one marker shape, a value twice as large then gets twice the area.

Worked example Figure 20 adds sepal length as marker area. Circles, triangles, and squares have different area constants for the same size setting, so compare sizes only within one species. The circular size key calibrates circles only. R and Python also use different physical units for marker size. Exact comparisons across shapes would need one common shape or a separate key for each shape.

R

```r
p_size <- p_scatter + aes(size = Sepal.Length) +
  scale_size_area(
    max_size = 5,
    limits = c(0, max(d$Sepal.Length)),
    breaks = c(4.5, 6, 7.5),
    name = "Sepal length (cm)")
```

Python

```python
fig, ax = plt.subplots()
maximum = d["Sepal.Length"].max()
sns.scatterplot(
    data=d, x="Petal.Length", y="Sepal.Width",
    hue="Species", style="Species",
    hue_order=species, style_order=species,
    markers=markers, palette=colours,
    size="Sepal.Length", size_norm=(0, maximum),
    sizes=(0, 120), alpha=.7, ax=ax)
ax.set(xlabel="Petal length (cm)",
       ylabel="Sepal width (cm)")
```

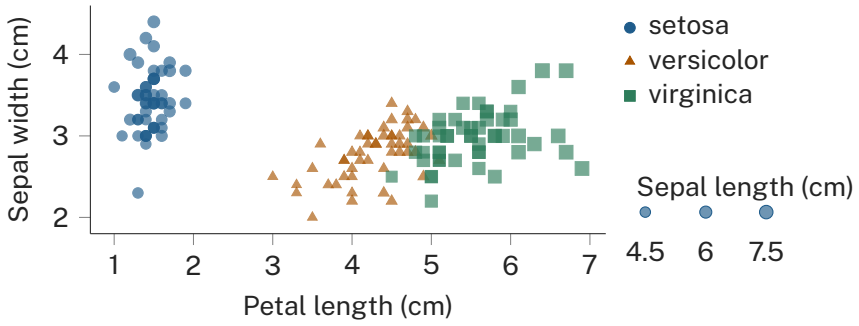


Figure 20. Petal length, sepal width, species, and sepal length in one scatter plot. The circular size key is in centimetres; proportional area comparisons apply within one marker shape.

The extra variable costs the reader attention. If exact comparisons of sepal length matter, give it its own aligned plot or facets instead.

7.3 Pooled and within-group correlation

What it is for Pearson's correlation coefficient r condenses the linear association in a scatter plot into one number.

The formula For nonconstant paired variables,

$$r = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2 \sum_i (y_i - \bar{y})^2}}.$$

It runs from -1 to $+1$ and has no unit. A value near zero does not rule out a nonlinear relationship. A large absolute value does not establish causation, does not rule out outliers or group differences, and can be driven by a few unusual observations. Always go back to the scatter plot and to how the data were collected.

Worked example Table 3 gives r for petal length and sepal width. All three within-species values are positive, while the pooled value is negative. The pooled value mixes the differences between species with the variation within each species. This reversal is Simpson's paradox [15]. Pooling is not a neutral substitute for a grouped question.

Table 3. Correlation of petal length with sepal width, pooled and within each species.

Group	n	r(petal length, sepal width)
All flowers	150	−0.428
Setosa	50	+0.178
Versicolor	50	+0.561
Virginica	50	+0.401

R

```r
print(cor(d$Petal.Length, d$Sepal.Width))
by_species <- by(d, d$Species, function(g)
  cor(g$Petal.Length, g$Sepal.Width))
print(by_species)
correlations <- cor(d[numeric_names])
print(round(correlations, 3))
```

Python

```python
print(d["Petal.Length"].corr(d["Sepal.Width"]))
by_species = {}
for s, g in d.groupby("Species", observed=True):
    by_species[s] = g["Petal.Length"].corr(
        g["Sepal.Width"])
print(by_species)
correlations = d[numeric_names].corr()
print(correlations.round(3))
```

For another dataset, identify the eligible paired records and report their number. Pairwise handling of missing values can give each coefficient a different set of records, and a correlation matrix should not hide those changes in denominator. Iris has no missing measurements.

8 Several variables in one overview

8.1 Scatter-plot matrices

What it is for A scatter-plot matrix draws every pair of numeric variables at once, to choose which pairs to study.

The counting rule For k variables the matrix has k^2 panels. The k diagonal panels show one variable each. The $k(k-1)/2$ distinct pairs each appear twice, once on each side of the diagonal with the axes exchanged.

Worked example With the four Iris measurements there are $4 \times 4 = 16$ panels, 4 on the diagonal and $4(4-1)/2 = 6$ distinct pairs. Only $4(4+1)/2 = 10$ panels are unique, but the full matrix still draws 16. Both languages select the same four numeric columns. Species defines groups; it is not a fifth numeric axis.

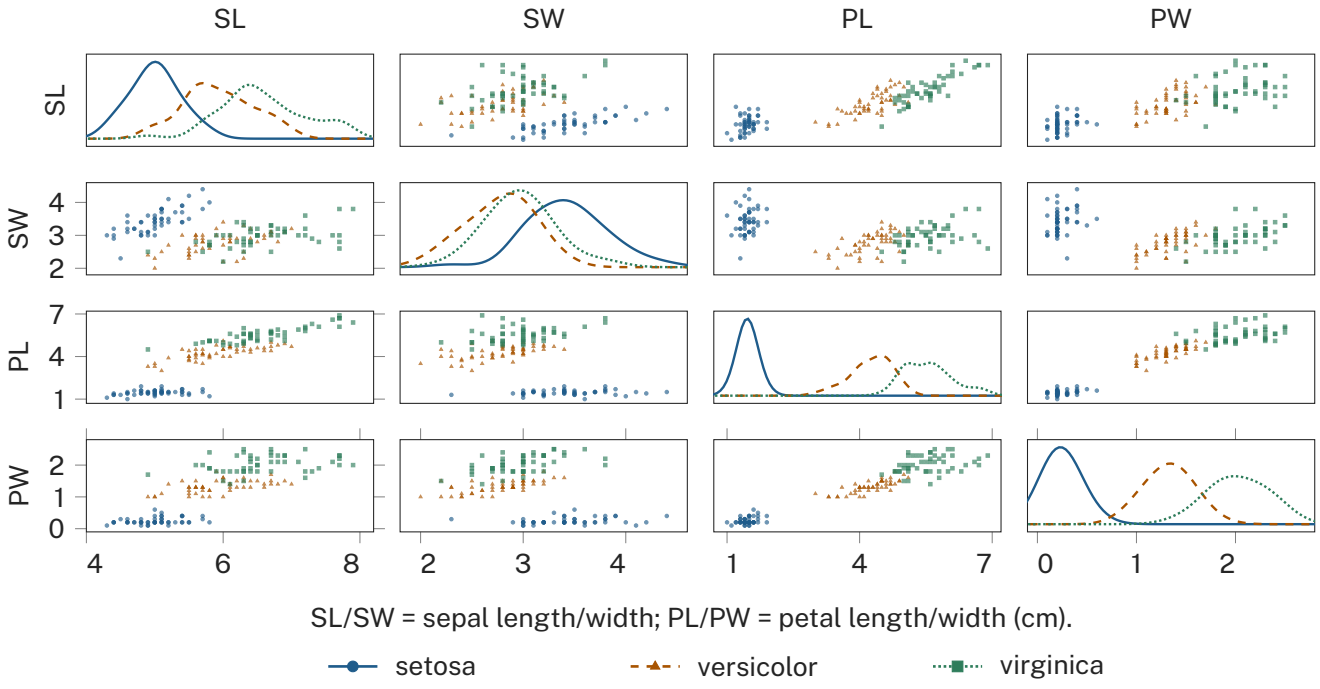


Figure 21. Four-variable Iris matrix. In off-diagonal panels, row labels identify the vertical variable and column labels the horizontal variable. Diagonal vertical axes show within-species density instead. Outer ticks give measurement scales in centimetres; colour, shape, and line style identify species.

R

```r
p_matrix <- ggpairs(d, columns = 1:4,
  mapping = aes(colour = Species,
                shape = Species),
  upper = list(continuous = "points"),
  lower = list(continuous = "points"),
  diag = list(continuous = wrap(
    "densityDiag", bw = h_group))) +
  scale_colour_manual(values = colours) +
  scale_shape_manual(values = c(16, 17, 15))
```

Python

```python
matrix = sns.pairplot(
    d, vars=numeric_names, hue="Species",
    palette=colours, markers=markers,
    diag_kind="kde",
    diag_kws={"bw_method": h_factor,
              "common_norm": False})
```

GGally's `ggpairs` can put summaries such as correlations in the upper triangle [12]. The example asks for points in both triangles to match the figure, and neither triangle should get a duplicate layer of points from a later call. A matrix is an overview. Enlarge a pair before making a detailed claim about it.

8.2 Correlograms

What it is for A correlogram shows a whole correlation matrix as colour, to spot strong and weak linear associations quickly.

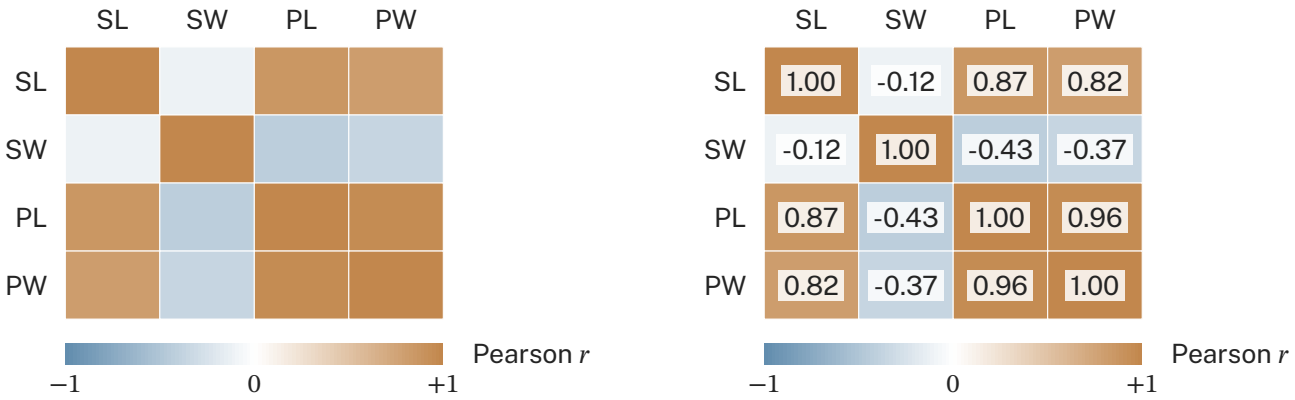


Figure 22. One computed correlation matrix shown as colour and as colour with values. SL, SW, PL, and PW mean sepal length, sepal width, petal length, and petal width. Colour uses the same bounded numerical scale in both panels.

Table 4. The six unique pooled Iris correlations, all based on 150 paired observations.

Variable pair	Pearson r
Sepal length / Sepal width	−0.118
Sepal length / Petal length	0.872
Sepal length / Petal width	0.818
Sepal width / Petal length	−0.428
Sepal width / Petal width	−0.366
Petal length / Petal width	0.963

The rules

- The diagonal is one, because a nonconstant variable is perfectly correlated with itself.
- The matrix is symmetric, because $r(x, y) = r(y, x)$.

- Use a diverging colour scale with fixed limits -1 and $+1$. This separates the signs and keeps different matrices comparable, even when the libraries use different palettes.
- Colour is coarse, so print the values in the cells or give a table.

Worked example Figure 22 and Table 4 show the six unique Iris correlations. Petal length and petal width have $r = 0.963$, printed to two decimals as 0.96. Rounding to 1.0 would suggest a perfect correlation when some variation remains.

R

```r
cells <- as.data.frame(as.table(correlations))
p_correlogram <- ggplot(cells,
  aes(Var1, Var2, fill = Freq)) + geom_tile() +
  geom_text(aes(label = sprintf("%.2f", Freq))) +
  scale_fill_gradient2(
    low = "#628DAE", mid = "white",
    high = "#C2874D", limits = c(-1, 1)) +
  coord_equal() +
  labs(x = NULL, y = NULL, fill = "r")
```

Python

```python
fig, ax = plt.subplots()
palette = sns.blend_palette(
    ["#628DAE", "white", "#C2874D"],
    as_cmap=True)
sns.heatmap(
    correlations, annot=True, fmt=".2f",
    vmin=-1, vmax=1, center=0, square=True,
    cmap=palette, ax=ax)
```

A correlogram hides nonlinearity, individual outliers, and group patterns such as the one in Table 3. Use it to choose relationships to inspect, then return to the scatter plots.

8.3 Parallel coordinates

What it is for Parallel coordinates show each row as one profile across all the numeric columns [9].

The rule Put the variables on adjacent vertical axes and join each row's values with a line. Columns in different units are first standardized one column at a time,

$$z_i = \frac{x_i - \bar{x}}{s_x},$$

where $\bar{x}$ and s_x are that column's mean and sample SD (divisor $n-1$). The vertical axis is then unitless, and zero is the column mean.

Worked example Figure 23 draws all 150 flowers, one line each, across the four standardized measurements.

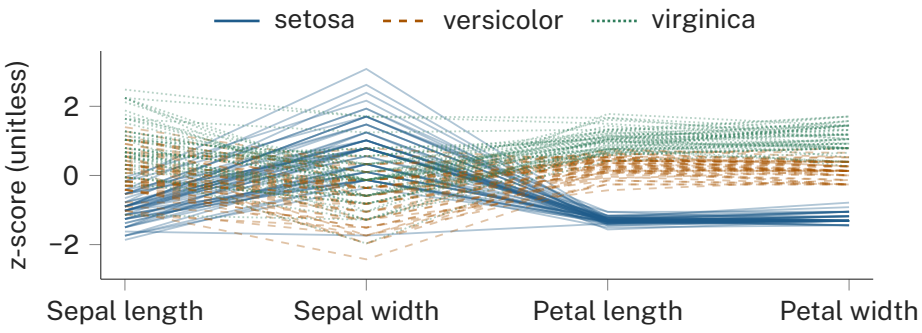


Figure 23. All 150 standardized flower profiles. Colour identifies species; values are sample-standard-deviation units from each column's mean. The four-axis order is explicit.

R

```r
z <- scale(d[numeric_names])
n <- nrow(d)
variable <- factor(numeric_names,
                   levels = numeric_names)
long <- data.frame(Row = rep(seq_len(n), 4),
  Variable = rep(variable, each = n),
  Z = as.vector(z), Species = rep(d$Species, 4))
p_parallel <- ggplot(long, aes(Variable, Z,
  group = Row, colour = Species,
  linetype = Species)) + geom_line(alpha = .35) +
  scale_colour_manual(values = colours) +
  scale_linetype_manual(values = styles) +
  labs(x = NULL, y = "z-score (unitless)")
```

Python

```python
z = d[numeric_names]
z = (z - z.mean()) / z.std(ddof=1)
fig, ax = plt.subplots()
for s, c, line in zip(species, colours, styles):
    rows = z.loc[d["Species"] == s].to_numpy()
    ax.plot(range(4), rows.T, color=c,
            linestyle=line, alpha=.35)
    ax.plot([], [], color=c, linestyle=line,
            label=s)
ax.set_xticks(range(4), numeric_names)
ax.set_ylabel("z-score (unitless)")
ax.legend()
```

R's `scale` and pandas' `std(ddof=1)` both use the sample SD. A population SD would give slightly different coordinates. A constant column has SD zero, cannot be divided by it, and needs separate treatment. Standardizing does not remove outliers or make every variable equally useful. Reordering the axes changes which crossings are visible, and crossings are not causal paths between variables.

9 Change over time

What it is for A line plot shows how one variable changes across ordered periods, such as reporting years.

The rule Put time on the horizontal axis and join each value to the next period. The line claims that adjacent periods belong to one series, so it does not suit unordered categories, and it is not a scatter plot of two measured variables.

Missing periods A missing report is not a measured zero. Keep the period in the table with a missing value, so the line breaks there.

- Dropping the row would draw a straight segment across the gap and imply a value that was never reported.
- Replacing it with zero would invent a sharp fall.

State in the caption which periods are missing, and why when that is known.

Worked example Figure 24 shows ten illustrative curbside-waste tonnages for the reporting years 2015–16 to 2024–25, with 2019–20 not reported. The line breaks at 2019–20 instead of joining 2018–19 to 2020–21.

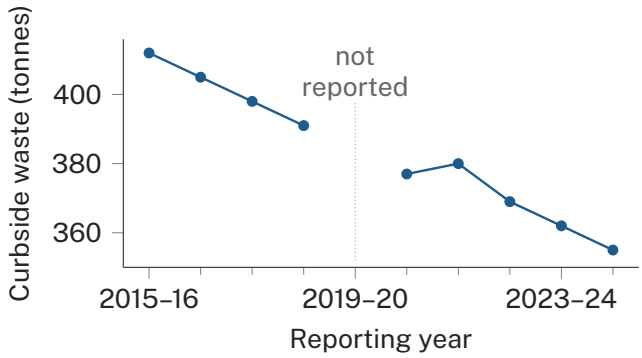


Figure 24. Ten illustrative annual curbside-waste tonnages with 2019–20 not reported. The line breaks at the missing year instead of dropping to zero or joining 2018–19 to 2020–21.

R

```r
# Illustrative values; 2015 means 2015-16.
annual <- data.frame(Year = 2015:2024,
  Tonnes = c(412, 405, 398, 391, NA,
             377, 380, 369, 362, 355))
starts <- seq(2015, 2023, by = 2)
p_line <- ggplot(annual, aes(Year, Tonnes)) +
  geom_line() + geom_point() +
  scale_x_continuous(breaks = starts,
    labels = sprintf("%d-%02d", starts,
                     (starts + 1) %% 100)) +
  labs(x = "Reporting year",
       y = "Curbside waste (tonnes)")
```

Python

```python
# Illustrative values; 2015 means 2015-16.
annual = pd.DataFrame({
    "Year": range(2015, 2025),
    "Tonnes": [412, 405, 398, 391, np.nan,
               377, 380, 369, 362, 355]})
fig, ax = plt.subplots()
ax.plot(annual["Year"], annual["Tonnes"],
        marker="o", color="black")
starts = range(2015, 2025, 2)
labels = [f"{y}-{(y + 1) % 100:02d}"
          for y in starts]
ax.set_xticks(starts, labels)
ax.set(xlabel="Reporting year",
       ylabel="Curbside waste (tonnes)")
```

ggplot2's `geom_line` and matplotlib's `plot` both break the line at `NA` or `NaN`. R also warns that `geom_point` removed one row. That warning reports the missing year and is not an error. Use the source's reporting-year labels, and choose a vertical range that shows the variation without implying a baseline the data do not have.

10 Choose, critique, and explain a chart

What it is for This checklist applies to any chart in this class, your own or one you read.

A chart chooser such as Abela's offers candidate displays [1]. Perception research explains why aligned positions and clear grouping make comparisons easier [3]. Together they give four questions to ask before publishing a figure.

1. What comparison should the audience make?
2. Do the variables, units, grouping, and denominator support that comparison?
3. Could binning, smoothing, ordering, or axis limits change the conclusion?
4. Can a reader identify the groups and reproduce the displayed values?

There is no precision ladder from histogram to density to ECDF to Q-Q plot. These charts answer different questions. Grouped boxes and violins trade a compact summary against estimated shape, and neither is always better. For proportions, aligned bars usually support close comparisons. For associations, look at the points and the groups before reducing them to a number.

When you critique a published chart, identify its question, data source, units, and denominator, and credit the chart and its source. A design critique is not a claim that the underlying data are wrong.

11 Summary

- Start from the question. The chart family follows from whether you are describing a distribution, a proportion, an association, or change over time.
- Every distribution chart depends on a convention (quartile rule, bin edges, bandwidth, or plotting positions). State it, and set it explicitly so R and Python agree.
- A compact summary can hide shape. Look at the observations alongside any box plot or correlation.
- Groups change the picture. Stacks show totals, normalized densities and ECDFs show shape, and a pooled correlation can have the opposite sign to every within-group one.
- A proportion needs its denominator, and a line needs its missing periods kept visible.

References

[1] A. Abela. *Chart Suggestions: A Thought-Starter*. Chart chooser. 2009. URL: `https://extremepresentation.com/wp-content/uploads/choosing-a-good-chart-09-1.pdf`.

[2] W. S. Cleveland and R. McGill. “Graphical perception: theory, experimentation, and application to the development of graphical methods”. In: *Journal of the American Statistical Association* 79.387 (1984), pp. 531–554.

[3] S. Few. “Data visualization for human perception”. In: *The Encyclopedia of Human-Computer Interaction*. 2nd ed. Interaction Design Foundation.

[4] R. A. Fisher. “The use of multiple measurements in taxonomic problems”. In: *Annals of Eugenics* 7.2 (1936), pp. 179–188.

[5] D. Freedman and P. Diaconis. “On the histogram as a density estimator: L_2 theory”. In: *Zeitschrift für Wahrscheinlichkeitstheorie und Verwandte Gebiete* 57.4 (1981), pp. 453–476.

[6] C. Gillespie, S. Locke, R. Davies, and L. D’Agostino McGowan. *datasauRus: Datasets from the Datasaurus Dozen*. Version 0.1.9. CRAN. 2025.

[7] J. L. Hintze and R. D. Nelson. “Violin plots: a box plot-density trace synergism”. In: *The American Statistician* 52.2 (1998), pp. 181–184.

[8] R. J. Hyndman and Y. Fan. “Sample quantiles in statistical packages”. In: *The American Statistician* 50.4 (1996), pp. 361–365.

[9] A. Inselberg. “The plane with parallel coordinates”. In: *The Visual Computer* 1.2 (1985), pp. 69–91.

[10] J. Matejka and G. Fitzmaurice. “Same stats, different graphs: generating datasets with varied appearance and identical statistics through simulated annealing”. In: *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*. 2017, pp. 1290–1294.

[11] R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing. Vienna, Austria, 2026.

[12] B. Schloerke, D. Cook, J. Larmarange, F. Briatte, M. Marbach, E. Thoen, A. Elberg, and J. Crowley. *GGally: Extension to ggplot2*. Documentation for `ggpairs`, `ggcorr` and `ggparcoord`. CRAN.

[13] D. W. Scott. *Multivariate Density Estimation: Theory, Practice, and Visualization*. Wiley, 1992.

[14] B. W. Silverman. *Density Estimation for Statistics and Data Analysis*. Chapman and Hall, 1986.

[15] E. H. Simpson. “The interpretation of interaction in contingency tables”. In: *Journal of the Royal Statistical Society, Series B (Methodological)* 13.2 (1951), pp. 238–241.

[16] H. A. Sturges. “The choice of a class interval”. In: *Journal of the American Statistical Association* 21.153 (1926), pp. 65–66.

[17] J. W. Tukey. *Exploratory Data Analysis*. Reading, MA: Addison-Wesley, 1977.

[18] H. Wickham, D. Navarro, and T. L. Pedersen. *ggplot2: Elegant Graphics for Data Analysis*. 3rd ed. Online edition, work in progress, `https://ggplot2-book.org/`.

[19] M. B. Wilk and R. Gnanadesikan. “Probability plotting methods for the analysis of data”. In: *Biometrika* 55.1 (1968), pp. 1–17.